

Reviewer Pack — Spectral Mahler Measure of Tseitin Moment Matrix Bounds SOS ...

Entry 146b8cf9b370 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-07 20:05:08 UTC

Spectral Mahler Measure of Tseitin Moment Matrix Bounds SOS Refutation

Entry ID: 146b8cf9b370 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-07 20:05:08 UTC

1. Conjecture

Field A (mathematical branch): Mahler measure / logarithmic heights of integer-spectrum matrices (analytic-Diophantine number theory) **Field B** (complexity object): Degree-2 SOS moment matrix for Tseitin Max-CUT on regular expanders

Statement:

Let $T(G,b)$ be an unsatisfiable Tseitin formula on a connected d -regular graph G ($d \geq 3$) with parity charges $b \in \{0,1\}^V$, and let $A(G,b)$ be the signed adjacency matrix with $A_{ij} = (-1)^{b_i + b_j}$ if $\{i,j\} \in E$ and 0 otherwise. Form the $(n+1) \times (n+1)$ canonical degree-2 SOS moment matrix $M(G,b)$ by setting $M_{0,0} = 1$, $M_{0,i} = 0$, $M_{i,i} = 1$, and $M_{i,j} = A_{ij}/(d-1)$ for $i,j \geq 1$; define the spectral Mahler measure $m(M) = \sum_{i: |\lambda_i| > 1} \log |\lambda_i|$ over eigenvalues of M . Conjecture: for every $(d, 2\sqrt{d-1})$ -spectral expander G and every parity-violating b ($\sum b_v \equiv 1 \pmod 2$), $m(M(G,b)) \geq \alpha_d \cdot n - O(\log n)$ with $\alpha_d = (1/2) \cdot \log((d-1)/(d-2)) > 0$, and any degree-2 pseudoexpectation certifying a $(0.878 + \varepsilon)$ -approximation to Max-CUT on G must have $m(M)$ strictly below $\alpha_d \cdot n/2$, so its existence is forbidden whenever the conjecture holds.

Rationale (proposer's reasoning):

Tseitin lower bounds against algebraic relaxations are typically proved via spectral expansion of an associated matrix, but spectral norm alone discards the multiplicative information encoded in the full eigenvalue sequence; the Mahler measure $m(M) = \log \prod \max(1, |\lambda_i|)$ is precisely the multiplicative invariant from Diophantine geometry (Lehmer's problem, height theory) that aggregates this information. Linking this height-theoretic invariant to SOS feasibility on Tseitin Max-CUT bridges a number-theoretic field essentially absent from proof complexity (5 prior papers) with the canonical SOS-vs-Tseitin separation question. Because $m(M)$ is a local function of the spectrum it neither algebrizes (it is not preserved under oracle relativization of M) nor falls to natural proofs (it is a property of the SOS object, not of a Boolean function family).

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2fe863c42b93cc89

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n \in \{10, 15, 20, 25, 30, 35, 40\}$ with 30 seeds each (210 odd-parity 3-regular instances), the conjecture is SUPPORTED iff (a) no instance has $m(M)/n < \alpha_3/2$ ($=0.1733$), and (b) the OLS slope of $m(M)$ on n satisfies $\text{slope} \geq \alpha_3$ ($=0.3466$) with 95% CI half-width ≤ 0.025 , and (c) $\geq 80\%$ of instances individually satisfy $m(M) \geq \alpha_3 \cdot n - c \cdot \log n$ for some fitted c .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -Tseitin formula expander Sum-of-Squares degree-2 lower bound Max-CUT -Mahler measure signed adjacency matrix eigenvalues Ramanujan expander -SOS moment matrix pseudoexpectation Tseitin spectral refutation

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def generate_random_graph(n, d):
    while True:
        edges = set()
        for i in range(n):
            neighbors = random.sample(range(n), d-1)
            if (i, *neighbors) not in edges and (i, *neighbors[::-1]) not in edges:
                edges.add((i, *neighbors))
        if len(edges) == n * d // 2:
            return [(j in neighbors) for j in range(n)] for _, *neighbors in edges]

def generate_parity_charges(n):
    return [random.randint(0, 1) for _ in range(n)]

def sign_matrix(A, b):
    n = len(b)
    M = [[0] * (n + 1) for _ in range(n + 1)]
```

```

M[0][0] = 1
for i in range(n):
    M[i+1][i+1] = 1
    for j in range(i+1, n):
        if A[i][j]:
            M[i+1][j+1] = (-1) ** (b[i] + b[j])
            M[j+1][i+1] = M[i+1][j+1]
return M

def eigenvalues(matrix):
    def gaussian_elimination(A, B):
        n = len(A)
        for i in range(n):
            max_row = i
            for j in range(i + 1, n):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            B[i], B[max_row] = B[max_row], B[i]
            for j in range(i + 1, n):
                factor = A[j][i] / A[i][i]
                for k in range(i, n):
                    A[j][k] -= factor * A[i][k]
                    B[j] -= factor * B[i]
        X = [0] * n
        for i in range(n - 1, -1, -1):
            X[i] = (B[i] - sum(A[i][j] * X[j] for j in range(i + 1, n))) / A[i][i]
        return X

    n = len(matrix)
    A = [[matrix[i][j] if i != j else 0 for j in range(n)] for i in range(n)]
    B = [sum(matrix[i][j] * matrix[j][k] for j in range(n)) for k in range(n)]
    eigenvals = []
    for _ in range(n):
        eigval, A, B = gaussian_elimination(A, B)
        eigenvals.append(eigval)
    return eigenvals

def spectral_mahler_measure(eigenvals):
    return sum(math.log(max(1, abs(eig))) for eig in eigenvals if abs(eig) > 1)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([10, 15, 20, 25, 30, 35, 40])
    G = generate_random_graph(n, 3)
    b = generate_parity_charges(n)
    M = sign_matrix(G, b)
    lambda_values = eigenvalues(M)
    m_M = spectral_mahler_measure(lambda_values)
    alpha_d = (1/2) * math.log(2)
    instances_tested = 1
    conjecture_holds = m_M >= alpha_d * n - 0.5 * math.log(n)
    counterexample = "" if conjecture_holds else f"m(M)/n={m_M/n} < alpha_d * n/2"
    return {
        "metric_name": "spectral_mahler_measure",
        "metric_value": m_M,
    }

```

```

        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_m_M = sum(r["metric_value"] for r in results) / len(results)
    std_m_M = math.sqrt(sum((r["metric_value"] - mean_m_M) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_m_M} std={std_m_M} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_m_M} std={std_m_M} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={m(M)/n < alpha_d * n/2} first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out after 240s (returncode=124) without producing any data, so none of the pre-registered support conditions could be evaluated. The critic also challenges, and no metric is available to confirm or refute the bound. | next: Optimize the eigenvalue computation (e.g., use sparse SciPy eigensolvers and reduce seed count or n-range) and rerun with a higher timeout to obtain the $m(M)/n$ statistics.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	155347
2	preregistration	claude_max	opus	0	0	8229
3	novelty	claude_max	opus	0	0	4001
4	novelty	claude_max	opus	0	0	8608
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	75590
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13757
7	judge	claude_max	opus	0	0	6062

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 271594 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/146b8cf9b370.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/146b8cf9b370.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/146b8cf9b370.tar.gz` (if generated)